



DETECTOR DE REPOSO, GOLPES Y CAÍDAS

APOLONIO VILLEGAS DIEGO ARMANDO

2020630050

APLICACIONES DE INTELIGENCIA ARTIFICIAL EN SISTEMAS

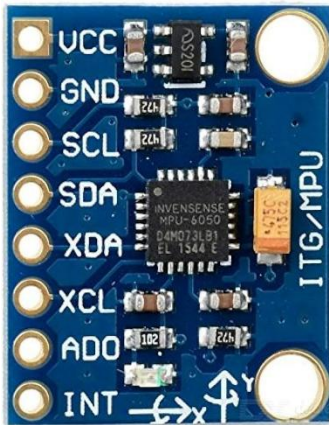
EMBEBIDOS

7BM1

INTRODUCCIÓN

Este reporte documenta el diseño e implementación de un detector de estado pasivo, golpes y caídas utilizando un Arduino Nano (ATmega328P) y un sensor MPU6050 por medio de un modelo de regresión logística. El objetivo es demostrar que se pueden desplegar modelos de inteligencia artificial en sistemas embebidos de recursos limitados. El modelo entrenado logró una exactitud del 94% por medio de dos evaluaciones efectuadas: validación cruzada y matriz de confusión.

MARCO TEÓRICO



El módulo MPU6050 es una unidad de medición inercial de 6 grados de libertad ya que combina un acelerómetro de 3 ejes y un giroscopio de 3 ejes.

Aceleración y acelerómetros

La aceleración es la variación de la velocidad por unidad de tiempo es decir razón de cambio en la velocidad respecto al tiempo

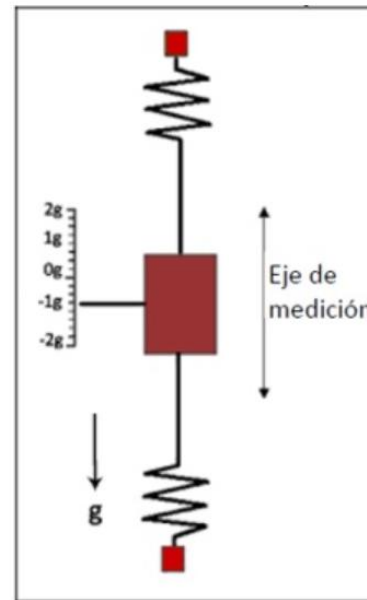
$$a = \frac{dV}{dt}$$

Así mismo la segunda ley de Newton indica que en un cuerpo con masa constante, la aceleración del cuerpo es proporcional a la fuerza que actúa sobre él mismo:

$$a = \frac{F}{m}$$

Este segundo concepto es utilizado por los acelerómetros para medir la aceleración. Los acelerómetros internamente tienen un MEMS (MicroElectroMechanical Systems) que de forma similar a un sistema masa resorte permite medir la aceleración.

Con un acelerómetro podemos medir esta aceleración, teniendo en cuenta que a pesar de que no exista movimiento, siempre el acelerómetro estará capturando la aceleración de la gravedad.



Regresión logística

La regresión logística es un algoritmo de aprendizaje supervisado en Machine Learning que generaliza la regresión lineal para poder hacer clasificación con ella. Para ello, la regresión logística comienza exactamente igual que la regresión lineal.

$$z_i = \vec{W}_i \cdot \vec{X} + b_i$$

El cambio en la regresión logística sucede en los pasos siguientes. El resultado z_i por sí solo no sirve para aplicar un ejercicio de clasificación, es necesario agregar una función que permita interpretar el valor como una probabilidad.

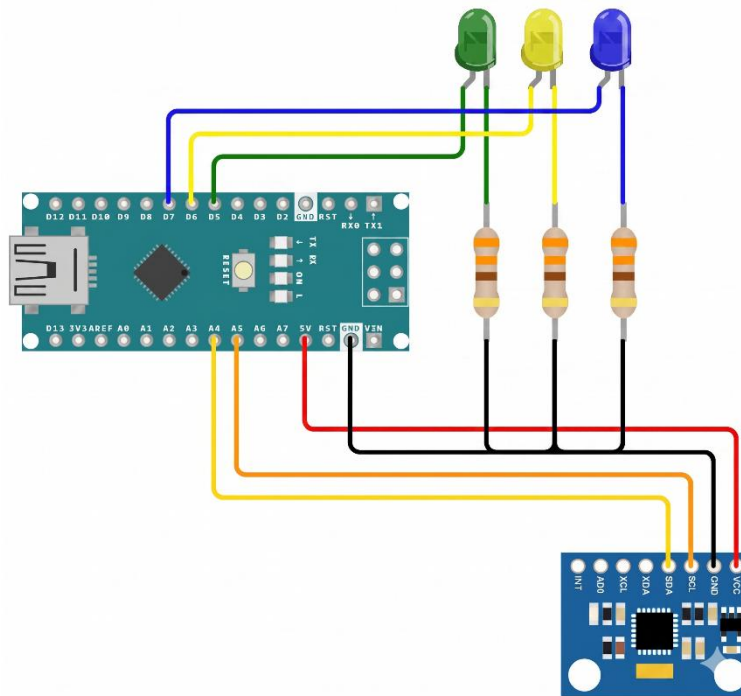
Para ello se utiliza la función *softmax*:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

Esta función genera la distribución de probabilidad sobre las k clases. $\text{softmax}(z_i)$ permite determinar la probabilidad de que z_i pertenezca a la i-ésima clase.

DESARROLLO

Conexión del circuito



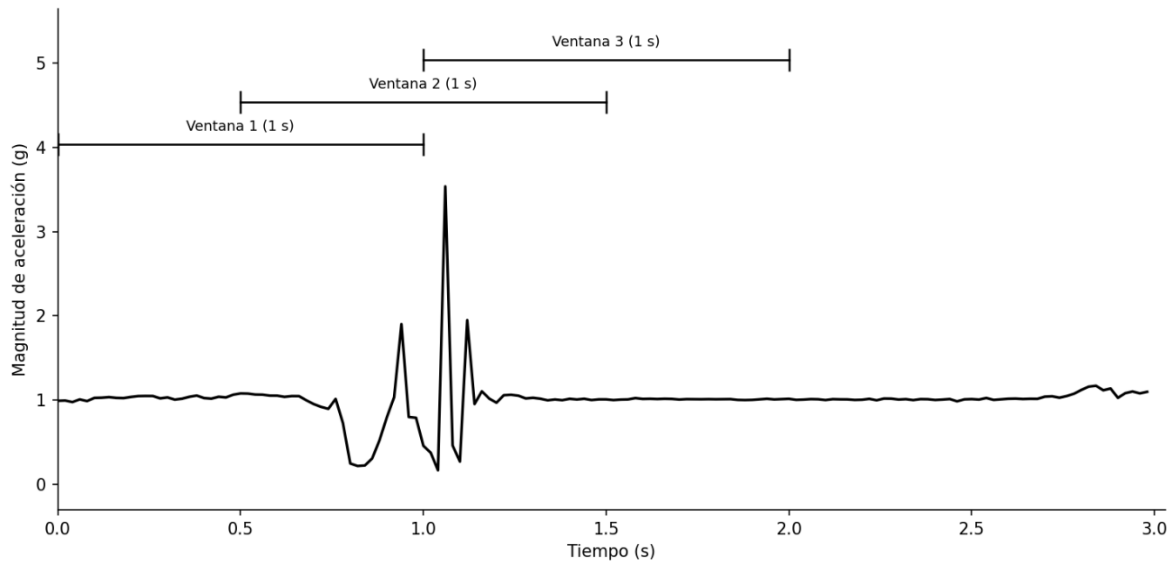
Para esta práctica únicamente vamos a utilizar el acelerómetro del sensor MPU6050 que hará comunicación por medio de I2C con el Arduino Nano, luego el microcontrolador enviará los datos a la computadora por medio de USART. Para indicar el comportamiento del circuito vamos a utilizar LEDs de la siguiente manera:

- LED verde: estado pasivo
- LED amarillo: un golpe
- LED azul: una caída

NOTA: se considera estado pasivo al acto de mantener el circuito en la mano sin movimientos bruscos o posado sobre una superficie.

Configuración del muestreo

La frecuencia de muestreo elegida es de 50Hz , es decir, una muestra cada 20ms que se logra configurando el Timer1 del microcontrolador en modo CTC. Para llevar a cabo la implementación del modelo de inferencia se considerará una ventana de 1 segundo que comprende 50 muestras. El desplazamiento de la ventana será de medio segundo, esto para que un evento no quede dividido en dos segundos distintos.



Captura del conjunto de datos

Para obtener el conjunto de datos que nos permitirá entrenar el modelo de inferencia se generaron tres archivos: `reposo.csv`, `golpes.csv` y `caídas.csv`, para lograrlo se utilizó el comando:

```
~$ tio /dev/ttyUSB0 -b 115200 --log --log-file [nombre del archivo].csv
```

Que vuelca las muestras hechas por el Arduino Nano con el programa `captura.c` en el archivo correspondiente, la captura de los datos para cada tipo de evento se llevó a cabo durante dos minutos cada uno.

Para el estado de reposo se sostuvo el circuito en la mano, cambiándolo de orientación luego de unos segundos, incluyendo posarlo en una superficie firme. Para el evento de golpe se sostuvo en la mano la protoboard y se golpeó con diferentes intensidades en distintos lugares. Finalmente, para el evento de caída se elevaba el circuito a diferentes alturas y se dejaba caer sobre una superficie firme.

Los tres archivos tuvieron que pasar por un proceso sencillo de limpieza, pues al inicio del volcado de los datos se incluían datos basura producto del handshake entre el Arduino Nano y el puerto USB de la computadora. Además, en ocasiones se imprimía múltiples veces los nombres de las cabeceras (`ax`, `ay`, `az`), así que se debían borrar las entradas extras para dejar únicamente una fila de nombres de encabezados. Por último, a lo largo del volcado de datos se colaban muestras corruptas, así que se borraron.

Definición de las características

Las características utilizadas para el entrenamiento del modelo se definieron en función de una ventana de 1 segundo, es decir, un segmento de 50 muestras. Para ello, se calculó la magnitud de cada una de las muestras con la formula:

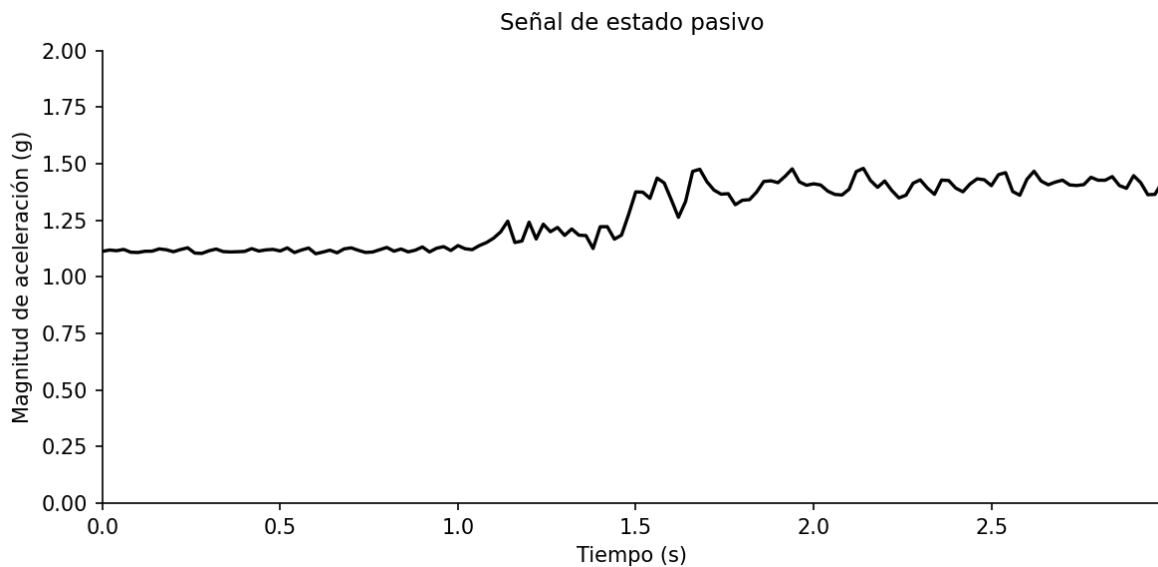
$$m = \sqrt{ax^2 + ay^2 + az^2}$$

Esto nos deja con 50 valores con los que definimos las características que utilizaremos para el modelo de inferencia:

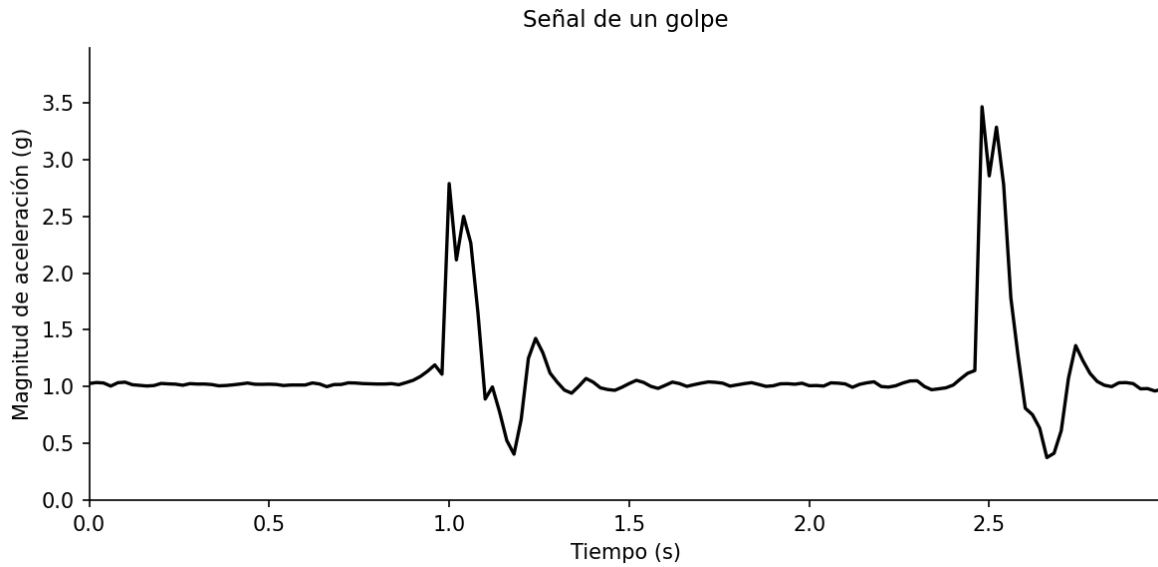
1. **Media:** el valor promedio de las 50 magnitudes
2. **Desviación estándar:** qué tan dispersas están las 50 magnitudes con respecto de su media
3. **Mínimo:** la magnitud con el valor más bajo
4. **Máximo:** la magnitud con el valor más alto
5. **Tiempo en valle:** el número de muestras que se encuentran por debajo del umbral de caída

La lógica utilizada para clasificar cada uno de los eventos es la siguiente:

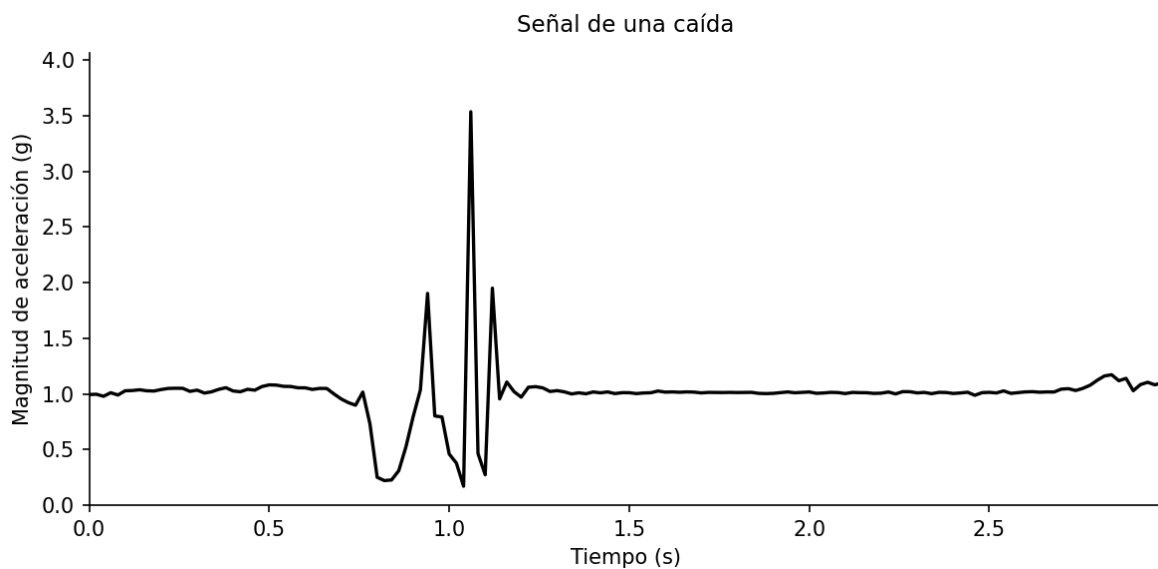
Estado pasivo: el cambio en el tiempo de una magnitud a otra es continuo, es decir, sin cambios bruscos de valor.



Golpe: en general, las magnitudes se mantienen en estado pasivo hasta que repentinamente hay un cambio brusco de valores entre ellas, y luego se vuelve al estado pasivo.



Caída: este es el evento más complejo pues se compone de más partes, primero las magnitudes se mantienen en estado pasivo, luego hay un pequeño cambio brusco entre ellas (el instante en el que se suelta el circuito), posteriormente las magnitudes de las muestras disminuyen a valores cercanos a cero (el momento donde el circuito se encuentra en caída libre), después cuando el circuito choca con la superficie las magnitudes de las muestras cambian abruptamente en el tiempo del mismo modo que un golpe, finalmente las magnitudes vuelven al estado pasivo.



Es importante recalcar en este punto que el momento donde las magnitudes de las muestras se acercan a cero justo antes de que sus valores cambien abruptamente producto del choque del circuito con la superficie es lo que diferencia un golpe de una caída. Este acercamiento de los valores al cero se debe a que, durante la caída libre, el acelerómetro

no experimenta una fuerza normal que lo sostenga. Recordemos que un acelerómetro no mide la aceleración del movimiento, sino la aceleración propia que produce la fuerza de soporte que actúa sobre él.

Como resultado de los pasos descritos hasta hora, se obtuvieron 353 ejemplos de los cuales:

- 175 corresponden al estado pasivo
- 91 son de golpes, y
- 87 son de caídas

Cada ejemplo consiste en sus valores para cada una de las 5 características y la etiqueta con su clase: estado pasivo, golpe o caída.

Entrenamiento del modelo

Antes de realizar el entrenamiento del modelo, los valores de las características de todos los ejemplos se normalizaron ya que se pueden presentar valores muy grandes -como la magnitud de una muestra al realizarse un golpe- o muy pequeños -como la magnitud de una muestra cuando se está en caída libre-, y el modelo daría importancia desproporcionada a los valores más altos. Para ello se utiliza la fórmula de la puntuación z :

$$z_i = \frac{x_i - \mu_i}{\sigma_i}$$

Tras esto, los valores de las características de todos los ejemplos quedan en una escala comparable. La media y la desviación estándar de cada característica se guardan por que el Arduino Nano también las utiliza para normalizar los valores con los que se lleva a cabo la inferencia.

Para hacer el entrenamiento del modelo se utilizó una regresión logística multinomial o más específicamente, LogisticRegression de la librería scikit-learn en el programa escrito en Python `entrenar.py` que internamente utiliza el descenso del gradiente hasta encontrar los valores de la matriz de pesos $\mathbf{W}$ y del vector de sesgos $\vec{b}$. Para evaluar el modelo se hicieron dos entrenamientos con propósitos distintos.

El primer entrenamiento se hizo con el propósito de determinar la estimación del rendimiento del modelo, para ello se hizo una validación cruzada con 5 particiones: el modelo se entrena con 4 particiones y se evalúa con la quinta, este proceso se repite 4 veces más rotando la partición con la que se evalúa en cada vez. El resultado de las 5 evaluaciones se promedió y fue de un 94.3% de exactitud, con una variación de $\pm 5.5\%$.

El segundo entrenamiento se hizo con el propósito de saber dónde se equivoca el modelo, para ello se generó una matriz de confusión: se apartó el 30% de los ejemplos como conjunto de prueba y con el 70% se entrenó el modelo. Con las inferencias hechas con el conjunto de prueba se construyó la matriz.

	Pasivo	Golpe	Caída
Pasivo	52	1	0
Golpe	1	25	1
Caída	0	1	25

Las filas indican la clase real, y las columnas las que se infirieron, es así que la diagonal son los aciertos. En total, el modelo predijo mal 4 ejemplos de los 106 utilizados como conjunto de prueba lo que indica un ~96% de exactitud.

Una vez evaluado y confirmado que el modelo funciona, se hizo un último entrenamiento con el 100% de los ejemplos para aprovechar todos los datos disponibles. Los pesos y sesgos obtenidos se guardan junto con la media y la desviación estándar de cada característica en el archivo `model.h`.

Inferencia

Para poner el modelo en producción se grabó en el microcontrolador el archivo `inferencia.c` que lee las 50 muestras por ventana de un segundo y que se desliza medio segundo adelante para evitar partir en dos un evento.

Por cada ventana se realizan los siguientes pasos:

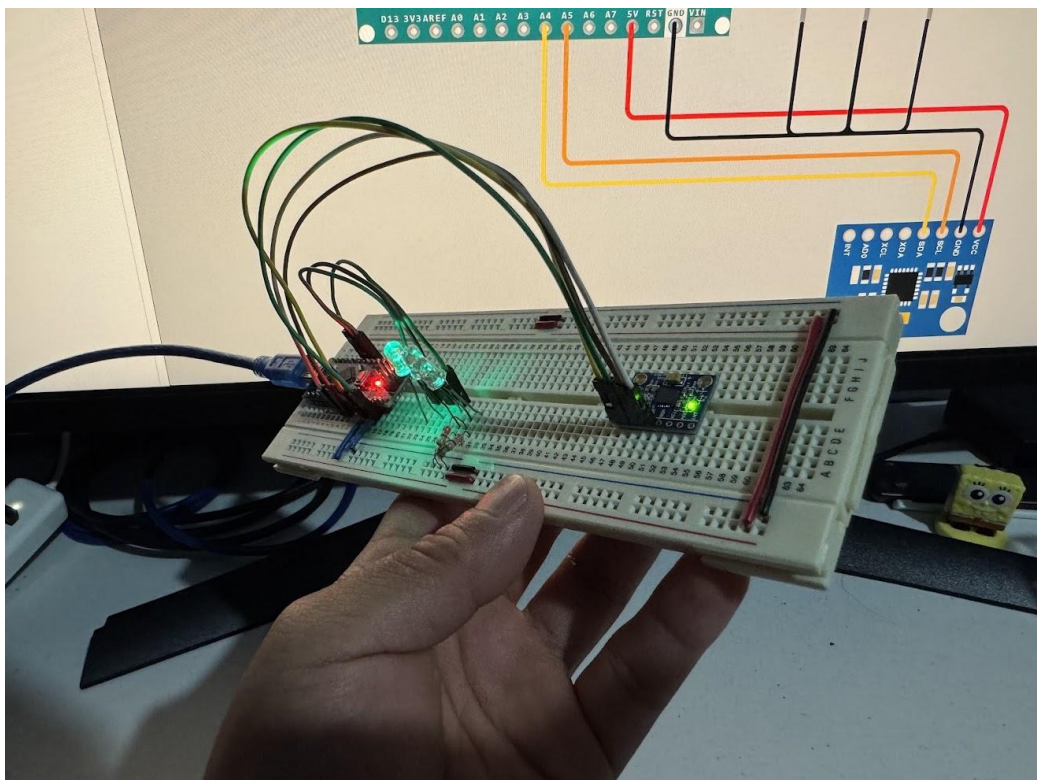
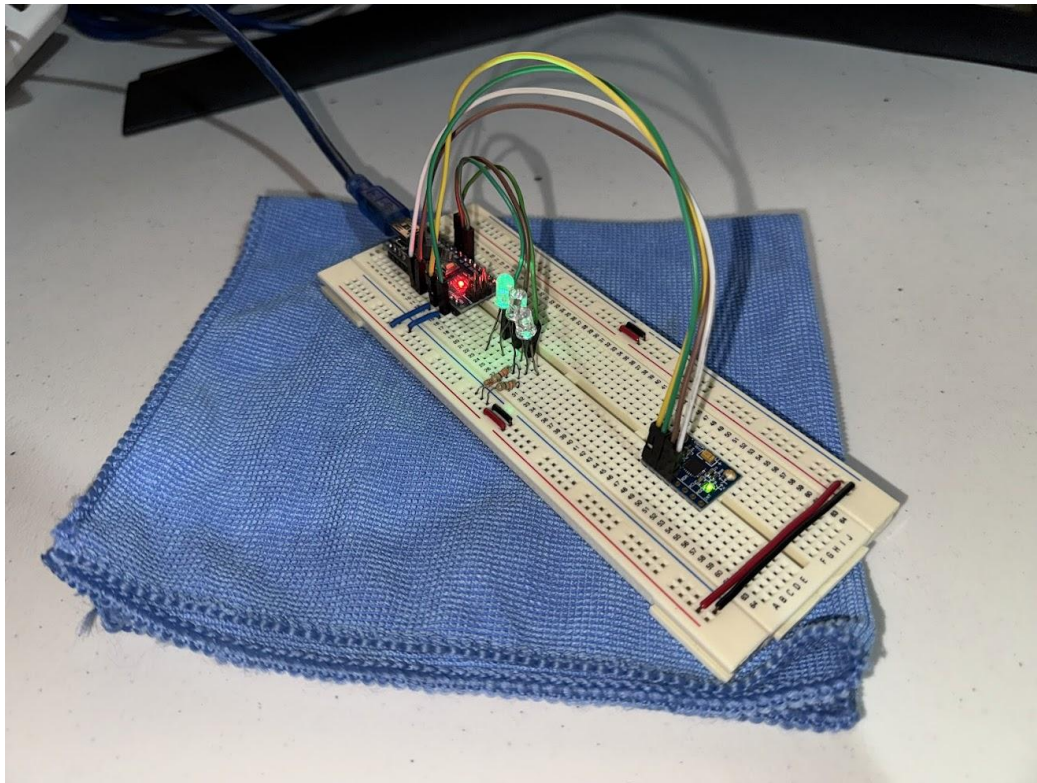
1. Se calculan los valores de las 5 características.
2. Se normalizan los valores obtenidos con MU y SD guardados en `model.h`.
3. Calcular el valor z de la ventana para cada clase con la fórmula $z_i = \vec{W}_i \cdot \vec{x} + b_i$
4. Se toma la clase con el valor z más alto como la predicción del modelo. En este caso no se aplica la función softmax pues calcular las exponenciales en un chip sin FPU tomaría bastantes ciclos de reloj, para esto nos apoyamos en el hecho de que un valor alto de z produce una probabilidad alta al aplicar softmax, lo mismo ocurre con valores medianos y bajos de z .
5. Dependiendo de la clase predicha se enciende el LED correspondiente: verde para estado pasivo, amarillo para golpe y azul para caída.

En el paso final el LED se mantiene encendido durante 1.5s para que se note, aunque el evento duró un instante. En el caso del evento golpe se espera 0.2s después de que ocurrió el evento para confirmar que lo que sigue no es un valle de caída libre, esto con el fin de no considerar el jalón al soltar el circuito para que caiga como un golpe.

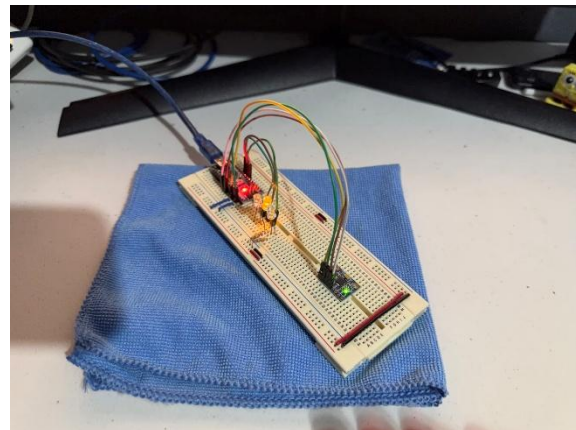
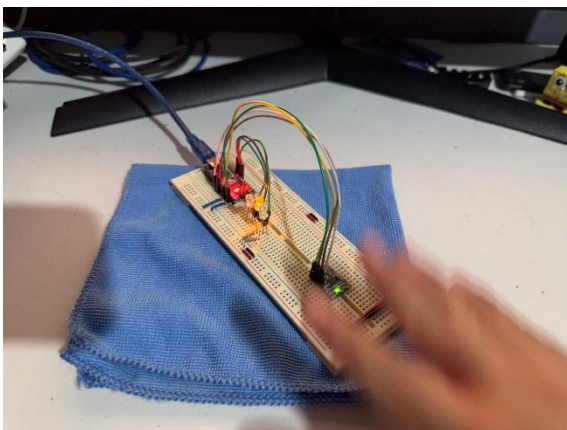
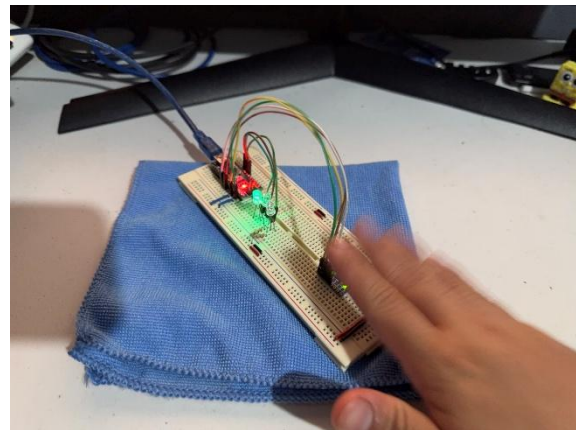
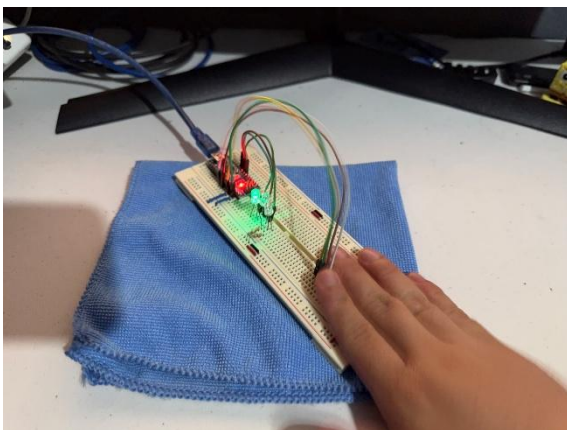
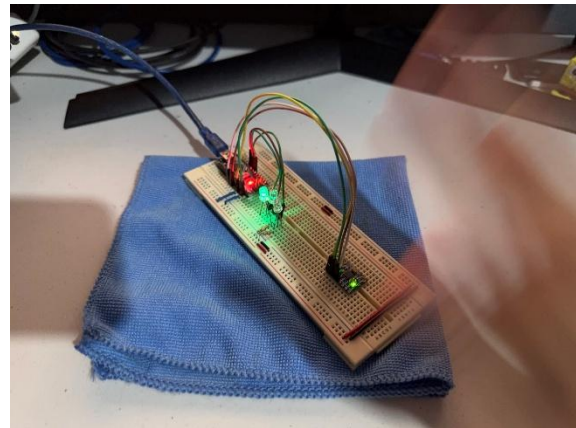
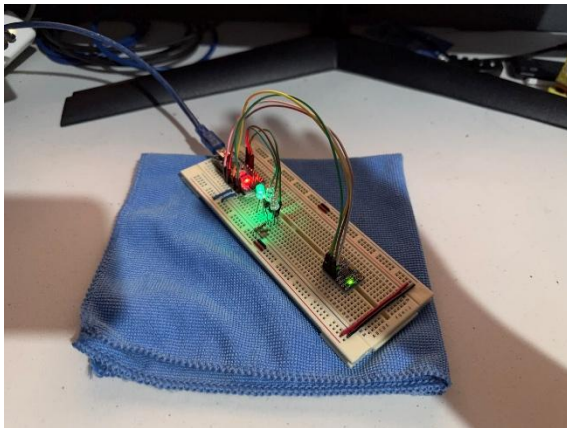
La ventana se desliza medio segundo adelante y se repite el proceso.

PRUEBAS

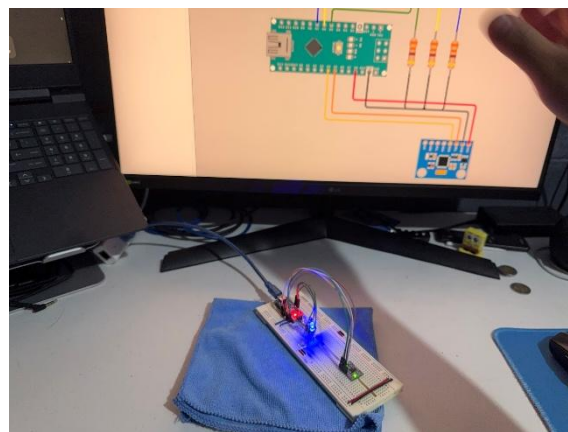
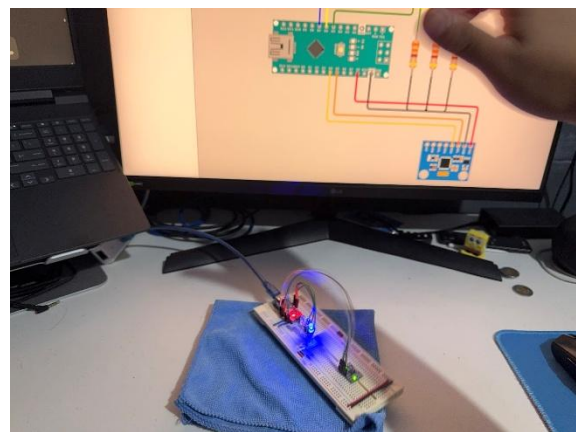
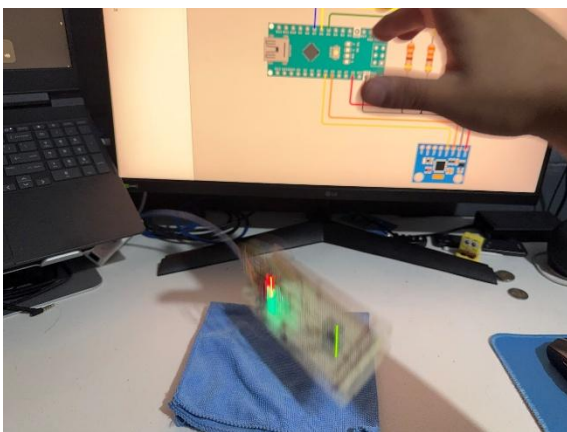
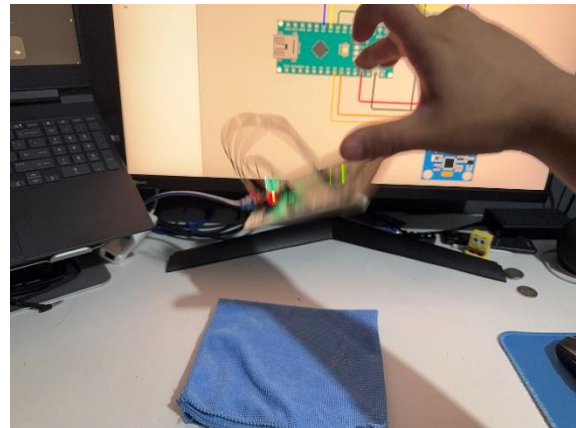
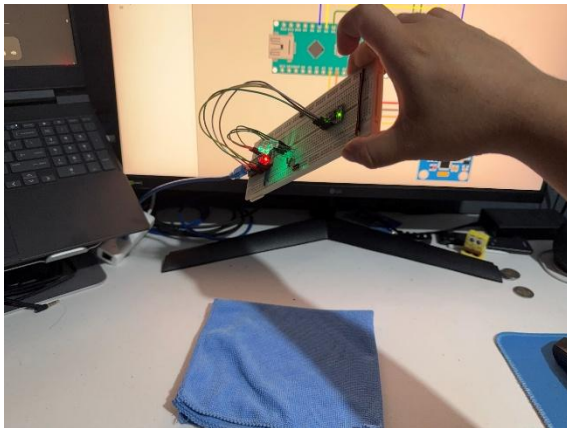
Estado pasivo



Evento de golpe



Evento de caída



CÓDIGO

model.h

```

1 #ifndef MODEL_H
2 #define MODEL_H
3
4 #define N_CLASES 3
5 #define N_FEATURES 5
6 #define VENTANA 50
7 #define UMBRAL_VALLE 3475L // umbral del valle de caída
8
9 /* media y desviación estándar de cada feature para normalizar los valores */
10 // media      desv. est.  mínimo      máximo      tiempo en valle
11 static const float MU[N_FEATURES] = {7932.6678f, 2383.3535f, 5181.4966f, 20360.5944f, 2.4703f};
12 static const float SD[N_FEATURES] = {1061.6796f, 2836.3707f, 3306.1666f, 15191.8946f, 3.9993f};
13
14 // Clases: 0 = caída, 1 = golpe, 2 = pasivo
15 static const float W[N_CLASES][N_FEATURES] = {
16 // media      desv. est.  mínimo      máximo      tiempo en valle
17 {+0.902198f, +1.935802f, -0.140186f, +0.467582f, +2.347043f}, // pasivo
18 {-0.643872f, -0.058061f, -1.612086f, +1.613189f, -2.261486f}, // golpe
19 {-0.258326f, -1.877741f, +1.752271f, -2.080771f, -0.085557f} // caída
20 };
21
22 // pasivo      golpe      caída
23 static const float B[N_CLASES] = {+0.120868f, +1.006492f, -1.127361f}; // sesgo de cada clase
24
25 #endif // MODEL_H

```

CONCLUSIONES

Me llevo una muy buena experiencia de esta práctica porque con ella por fin pude consolidar el proceso completo de implementación de un modelo de Machine Learning. Me di cuenta de que en realidad cualquier cosa que se pueda medir puede ser utilizado en un modelo de inferencia para diversos objetivos.

Por otra parte, definir muy bien las características o features que utilizará el modelo puede hacer la diferencia entre que este pueda hacer una buena clasificación o no, para este proyecto en específico, incluir el valor mínimo de las magnitudes y el número de muestras en el valle de caída libre fue lo que permitió al modelo poder hacer una buena clasificación entre golpe o caída -antes de eso, ambos se clasificaban igual-, en este sentido, los conocimientos básicos de física de mi curso de mecánica me permitieron poder llegar a entender por qué se presenta el valle en el evento de caída y en el golpe no.

Lo anterior me lleva al siguiente punto y es que es importante tener conocimiento básico del área en la que se va a aplicar la inteligencia artificial. En muchas ocasiones he escuchado que se menciona que la Inteligencia Artificial es una intersección entre matemáticas, computación y conocimiento del negocio; nunca había entendido verdaderamente el papel que desempeñan las reglas del juego hasta ahora.

También es imperativo que siempre se haga una evaluación exhaustiva de los modelos que se entrenan para saber si son lo suficientemente confiables para ser puestos en producción, para ello se tienen diversas técnicas además de las vistas aquí como Exactitud balanceada, Coeficiente de Matthews, Kappa de Cohen entre otras.

Finalmente me doy cuenta de que el área de Edge AI tiene bastante potencial, ya que actualmente con el apogeo de los Grandes Modelos de Lenguaje y el cómputo en la nube se destinan cantidades bestiales de recursos a mantener centros de datos lo que impacta directamente en el medio ambiente y en los recursos de hardware disponibles para particulares. Tener la posibilidad de desplegar pequeños modelos de detección de objetos o de reconocimiento de voz y sonido en hardware tan limitado como una Raspberry Pi Pico, o poder montar un LLM local cuantizado en una computadora modesta es el primer paso para democratizar el acceso a la Inteligencia Artificial para todos.

Gracias :)